Performance Insights on Amazon RDS for PostgreSQL

Now part of CloudWatch Database Insights: concepts, day-to-day use, and real-time troubleshooting

Version 1.0 | 29 October 2026

Read this first: what changed in 2026

Status as of 29 October 2026. AWS ended the standalone **Performance Insights** console experience on **31 July 2026** (the date was extended more than once). The old entry point now redirects to **Amazon CloudWatch Database Insights**. The ideas you know from Performance Insights (DB load, wait events, Top SQL, slicing by user and host) all still exist there, and the Performance Insights **API** continues to work unchanged. This guide uses both names: 'Performance Insights' for the concepts and API, and 'Database Insights' for the dashboard you open today.

Still the same	What is different now	What you should do
• DB load measured as average active sessions • Wait events and Top SQL • Slice by user, host, database, application • API (`aws pi ...`) and the `EnablePerformanceInsights` parameters, CloudFormation and Terraform settings	• Dashboard lives in CloudWatch Database Insights • Two modes: **Standard** (default) and **Advanced** • Long retention, guided lock analysis, OS-process view and fleet monitoring need Advanced (or paid retention in Standard) • On-demand analysis now requires Advanced, per AWS's notice	• Check each instance's mode (SU-02) • Choose Standard or Advanced on purpose (SU-01) • Update runbooks that say 'open Performance Insights' • Export history you must keep

Source note: this summary reflects AWS documentation and announcements reviewed on 29 October 2026. Retention options, prices and feature availability by engine change over time, so confirm against the current AWS documentation and pricing page before making cost or compliance decisions.

How to use this guide

- **Part 1** explains the concepts clearly: what DB load is, how to read the chart, wait events for PostgreSQL, and the two modes.
- **Part 2** is one-time setup (enable, settings, access, alarms).
- **Part 3** is the day-to-day routine: daily, weekly, release-day and monthly.
- **Part 4** is real-time troubleshooting: for each chart pattern, what to click, what to run, what it means, and how to fix it.
- **Part 5** covers problems with the tool itself, quick answers and a cheat sheet.

PART 1 | Understanding it

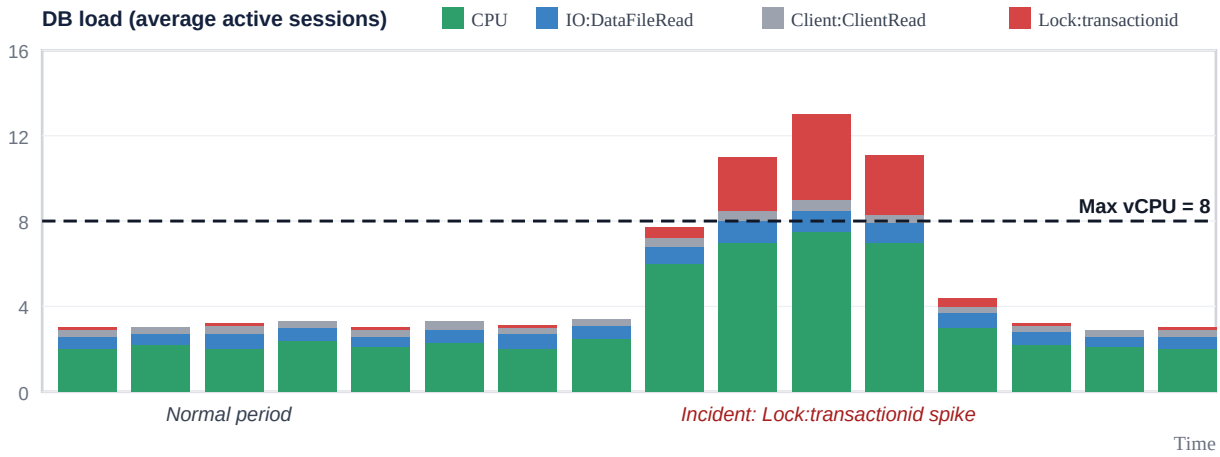
1.1 What it is and how it works

Performance Insights answers one question: **where is my database spending its time, and which SQL is responsible?** Once a second it looks at which sessions are active (running a query or waiting on something), records what each one is doing, and stores that history. The result is shown as a chart of **DB load** that you can slice by wait event, SQL statement, user, host, database and application. No agent is installed on your database server.

DB load = average active sessions (AAS). If the load is 4, then on average 4 sessions were busy at the same moment. Because it counts sessions rather than percentages, it shows what queries are doing, which CPU graphs cannot.

Why this beats CPU and other averages: CloudWatch CPU says the machine is busy but not why. DB load says how many sessions are busy, what they wait on, and which statements they run. It samples every second, so short spikes that one-minute averages smooth away remain visible.

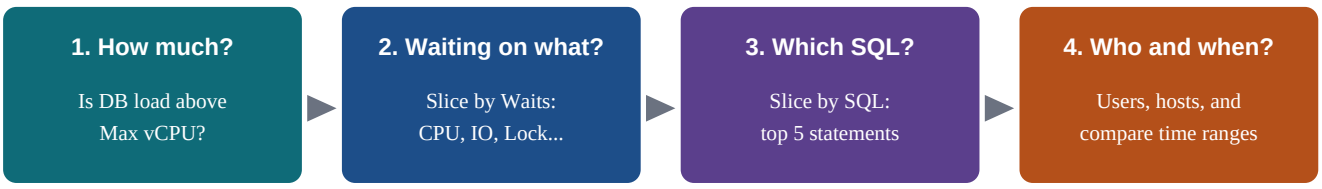
1.2 Reading the DB load chart



Illustrative example, not real data. Each bar is one time slice; each colour is a wait event; the dashed line is the number of vCPUs.

- **Height of the bar:** how many sessions were active. Taller means busier.
- **Colours:** what those sessions were doing (the wait event). `CPU` means running on, or waiting for, a CPU.
- **Dashed Max vCPU line:** the number of virtual CPUs. Sessions in the `CPU` colour above this line are queueing for a processor.
- **The incident above:** load rises far above 8, but the new colour is a **Lock** wait. More CPU would not fix that; the sessions are waiting for a lock, so you must find the blocker (LV-03).

The four-question method



Use the same four questions for every investigation. They keep you from guessing.

1.3 Common chart patterns and what they mean

What you see	Meaning	Next step
Load well below Max vCPU, mostly CPU	Healthy headroom	Nothing to do; record the baseline (DA-06)
Load at or above Max vCPU, mostly CPU	CPU saturated; sessions queue for processors	Find the top SQL by CPU; tune or scale (LV-01)
Load above Max vCPU, mostly IO or Lock colours	Sessions are waiting, not computing; the bottleneck is not CPU	Fix the wait: LV-02, LV-03, LV-06
One colour suddenly dominates	Single cause, usually one query, one blocker or one client	Slice by SQL, then Users/Hosts
Step change at a clock time	Deployment, parameter change, job start	Compare before and after (RL-01, LV-08)
Slow upward staircase over days	Data growth, stale statistics, creeping regression	Review top SQL trend and `ANALYZE` (WE-01)
Regular spikes at fixed intervals	Scheduled job or periodic process	Match times to cron/batch (LV-09)
Low load but users complain	Delay is outside the database	Check application, pool and network (LV-10)

1.4 Slicing the load: the dimensions

Slice by	Question it answers	Typical use
Waits	What are sessions waiting for?	Decides the kind of fix: index, lock, I/O, connection
SQL	Which statements cause the load?	Find heavy or regressed queries; build the tuning list
Users	Which database account?	Spot ad-hoc users or a new service account
Hosts	Which client machine?	Find the misbehaving application server
Databases	Which database in the instance?	Multi-database instances
Applications	Which application name?	Needs `application_name` set in connection strings

Tip: set `application_name` in every application's connection string. It makes the Applications slice and `pg_stat_activity` far more useful.

1.5 Wait events you will see on PostgreSQL

A wait event says what a session was waiting for at the moment it was sampled. These are the ones that matter most on RDS for PostgreSQL.

Wait event	What it means	Typical causes	First action
CPU	Session is running on a CPU or waiting for one (not a wait event in PostgreSQL itself; shown when no wait is reported)	Heavy queries, bad plans, missing indexes, too many concurrent sessions	Open Top SQL by CPU; tune or scale (LV-01)
IO:DataFileRead	Waiting to read a table/index page from storage because it is not in shared buffers	Sequential scans, missing indexes, cache too small, cold cache after restart	Find the SQL reading most blocks; add index; check memory (LV-02)
IO:BufFileRead / BufFileWrite	Reading/writing temporary files	Sorts and hashes that do not fit in `work_mem`	Tune query or raise `work_mem` for that role (LV-06)
IO:WALWrite	Waiting for WAL buffers to be written to the WAL file	Very high commit rate, slow storage	Batch commits, check write latency and IOPS (LV-06)
Lock:transactionid	Waiting for a row-level lock held by another transaction	Hot rows, long transactions, idle-in-transaction sessions	Find the blocker (LV-03)
Lock:relation	Waiting for a table-level lock	DDL (`ALTER TABLE`), `VACUUM FULL`, explicit `LOCK TABLE` behind a long query	Find and cancel the blocker or the DDL (LV-03)
Lock:tuple	Waiting for a lock on a single row version	Many sessions updating the same row	Reduce contention; shorten transactions

Wait event	What it means	Typical causes	First action
Lock:extend	Waiting to extend a relation (add pages)	Many concurrent inserts into one table/index	Batch inserts; partition; check I/O
Lock:advisory	Waiting for an application advisory lock	Application-level locking	Review the application's locking logic
LWLock:buffer_content	Waiting for other sessions finishing I/O on the same page	Hot pages, heavy concurrent writes	Spread writes; review hot indexes (LV-05)
LWLock:lock_manager	Contention on the lock manager when fast-path locks are not possible	Queries touching many tables/partitions/indexes at high concurrency	Reduce partitions/indexes per query; pooling (LV-05)
LWLock:pg_stat_statements	Contention updating the statements hash table	Very many distinct queries (for example unparameterised SQL)	Parameterise queries; review `pg_stat_statements.max`
Client:ClientRead	Database waiting for the client to send the next command/data	Idle sessions mid-transaction, slow application code, network latency, large COPY from client	Look at the application side (LV-04)
Client:ClientWrite	Database waiting to send results to the client	Huge result sets, slow client or network	Reduce rows returned; paginate
IPC:ProcArrayGroupUpdate	Coordinating transaction commits across sessions	Very high commit concurrency	Batch commits; review connection counts
Timeout:PgSleep	Session called `pg_sleep`	Application/test code	Check why it sleeps

1.6 Standard mode versus Advanced mode (RDS for PostgreSQL)

Capability	Standard (default)	Advanced
DB load chart, waits, Top SQL, per-query metrics	Yes	Yes
Retention of detailed database and per-query metrics	Free up to 7 days; paid retention of 1 to 24 months at the former Performance Insights price	Long retention included (the documented enable procedure uses at least 465 days, about 15 months)
Alarms and graphs on database metrics	Yes (7 days)	Yes
Guided lock analysis (lock tree)	No	Yes (supported for RDS for PostgreSQL)
OS-process analysis (needs Enhanced Monitoring)	No	Yes
Fleet-wide monitoring across databases	No	Yes
RDS events shown in CloudWatch	No	Yes
On-demand analysis	No (per AWS's July 2026 notice)	Yes
SQL execution plans	Not offered for RDS for PostgreSQL	Not offered for RDS for PostgreSQL (available for Aurora PostgreSQL, RDS for Oracle, RDS for SQL Server)
Price	No extra charge for 7-day tier	Per vCPU-hour (AWS material cites about $0.0125 per vCPU-hour; verify for your Region)

PART 2 | One-time setup

Do these once per instance (or bake them into your infrastructure code). They take little time and decide how useful the data will be later.

SU-01	Decide the mode: Standard or Advanced
When	Before enabling or reviewing monitoring on an instance. Standard is the default and free for up to 7 days of detailed history. Advanced adds long retention and extra tools.
Choose Standard if	You need the DB load chart, wait events and Top SQL for live troubleshooting and a week of history is enough (dev/test, small systems, cost-sensitive).
Choose Advanced if	You want months of history for trend and capacity analysis, the guided **Lock analysis** for RDS PostgreSQL, OS-process analysis, fleet-wide views across many databases, or RDS events inside CloudWatch. Typical for production and regulated workloads.
Watch out	On RDS for PostgreSQL, SQL execution-plan capture is **not** offered in either mode (it exists for Aurora PostgreSQL, RDS for Oracle and RDS for SQL Server). Use `EXPLAIN` and `auto_explain` for plans.

SU-02	Enable and verify on an instance (no downtime)
When	New instance, or an existing one that is not yet on the mode you want. Turning on the Advanced mode does not cause downtime.
Commands	<pre>``` # check what is enabled today aws rds describe-db-instances --db-instance-identifier <id> \ --query "DBInstances[0].[PerformanceInsightsEnabled,PerformanceInsightsRetentionPeriod, DatabaseInsightsMode,MonitoringInterval,DbiResourceId]" # Standard mode aws rds modify-db-instance --db-instance-identifier <id> --apply-immediately \ --enable-performance-insights --database-insights-mode standard # Advanced mode (the documented procedure requires a retention of at least 465 days) aws rds modify-db-instance --db-instance-identifier <id> --apply-immediately \ --database-insights-mode advanced --enable-performance-insights \ --performance-insights-retention-period 465 ```</pre>
Verify	The describe command shows `PerformanceInsightsEnabled = True` and the expected `DatabaseInsightsMode`. In the console the Database Insights dashboard for the instance shows data within a few minutes. Save `DbiResourceId` (starts with `db-`): the API calls use it.
Watch out	The older name 'Performance Insights' still appears in API and CLI parameters, CloudFormation and Terraform (for example `EnablePerformanceInsights`). That does not mean the retired console is in use; existing templates keep working. Advanced mode is billed per vCPU-hour, so confirm cost before enabling it fleet-wide.

SU-03 PostgreSQL settings that make the data useful

Why

The dashboard's per-query numbers come from PostgreSQL's own statistics. If the extension is missing or SQL text is truncated, you see load but cannot see what the queries are.

Check and set

```
-- 1) pg_stat_statements must be loaded and created
SHOW shared_preload_libraries;              -- should include pg_stat_statements
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- 2) SQL text length (static parameter: needs a reboot)
SHOW track_activity_query_size;

-- 3) I/O timing per query (dynamic, small overhead, very useful for IO waits)
SHOW track_io_timing;

# change in the parameter group (example)
aws rds modify-db-parameter-group --db-parameter-group-name <pg> \
  --parameters "ParameterName=track_io_timing,ParameterValue=1,ApplyMethod=immediate"
```

Verify

`SELECT count(*) FROM pg_stat_statements;` returns rows, and long statements show enough text in the dashboard to identify them.

Watch out

Raising `track_activity_query_size` increases shared memory per session slightly and needs a reboot. If you have tens of thousands of distinct unparameterised queries the statements table churns; parameterise queries in the application.

SU-04 Give people the right access

Why

SQL text can contain sensitive literals. Give broad read access to metrics and narrower access to SQL text.

Commands

```
# AWS managed policies for the Performance Insights API
#   AmazonRDSPerformanceInsightsReadOnly   (view dashboards and API data)
#   AmazonRDSPerformanceInsightsFullAccess (also create/modify)
aws iam attach-user-policy --user-name <dba_user> \
  --policy-arn arn:aws:iam::aws:policy/AmazonRDSPerformanceInsightsReadOnly

# Database Insights also supports fine-grained access control for sensitive dimensions
# such as SQL text: restrict by IAM policy on the pi: and cloudwatch: actions.
```

Verify

A read-only user can open the dashboard but cannot change the mode or retention; a restricted user cannot see SQL text.

Watch out

Developers often need SQL text to fix their own queries. Agree the policy with the security team instead of blocking it completely.

SU-05 Turn on Enhanced Monitoring (needed for OS process analysis)

Why

Advanced mode can show operating-system processes (CPU and memory per process) next to database load, but only if Enhanced Monitoring is on.

Command

```
aws rds modify-db-instance --db-instance-identifier <id> --apply-immediately \
  --monitoring-interval 60 --monitoring-role-arn <role_arn>
```

Verify

`MonitoringInterval` is 60 (or 1 to 15 for finer detail) and OS metrics appear.

Watch out

A 1-second interval gives fine detail but costs more in CloudWatch Logs. Use 60 s by default and 1 to 5 s only for production systems that need it.

SU-06	Alarm on database load
Why	DB load measured against the number of vCPUs is the single best 'is the database overloaded' signal. Alarm on it so you do not depend on someone watching the chart.

Commands	`# find the vCPU count of the instance class (db.m6g.large -> m6g.large)`
	`aws ec2 describe-instance-types --instance-types <m6g.large> \`
	`  --query "InstanceTypes[0].VCpuInfo.DefaultVCpus"`
	
	`aws cloudwatch put-metric-alarm --alarm-name rds-<id>-dbload-high \`
	`  --namespace AWS/RDS --metric-name DBLoad \`
	`  --dimensions Name=DBInstanceIdentifier,Value=<id> \`
	`  --statistic Average --period 60 --evaluation-periods 10 \`
	`  --threshold <vCPU_count> --comparison-operator GreaterThanThreshold \`
	`  --alarm-actions <sns_topic_arn>`

Verify	Trigger a test (or lower the threshold temporarily) and confirm the notification arrives. The metrics `DBLoad`, `DBLoadCPU` and `DBLoadNonCPU` are published in the `AWS/RDS` namespace.
Watch out	If the metric is missing, confirm Database Insights is enabled on the instance. Set the threshold from your own baseline (DA-06) rather than blindly at the vCPU count if your workload normally runs near it.

PART 3 | Day-to-day activities

A 15 to 20 minute routine each morning. It is read-only and safe on production. Open the dashboard for each important instance, and follow the four questions: how much, waiting on what, which SQL, who and when.

DA-01 Open the dashboard and judge overall load

Purpose	Answers the first question every day: is the database busier than it can handle? Look at the **DB load** chart for the last 24 hours.
Do	• Open CloudWatch Database Insights for the instance (the old Performance Insights entry redirects here). • Set the time range to 24 hours. • Compare the stacked load with the dashed **Max vCPU** line.
Healthy looks like	Load usually well below Max vCPU, short peaks that stay under or only briefly touch it, and a shape that repeats day to day (business hours high, night low).
If not	Sustained load above Max vCPU, new peaks, or a changed shape: continue with DA-02 and DA-03, then the matching live scenario (LV-01 to LV-10).

DA-02 Slice the load by Waits

Purpose	The colours in the chart are wait events. The biggest colour tells you what sessions spend their time on, which decides the type of fix.
Do	• Switch the slice to **Waits**. • Note the top two or three events and their share of the load. • Look up each in the wait-event table on the previous pages.
Healthy looks like	Mostly `CPU` with some `IO:DataFileRead` and `Client:ClientRead`, in proportions similar to yesterday. Almost no `Lock:*` or `LWLock:*`.
If not	A new or growing `Lock:*`, `IO:*` or `LWLock:*` band is the headline. Go to the scenario for that event.

DA-03 Find the top SQL

Purpose	Load is created by statements. Seeing the top few and whether they are the usual ones is the quickest way to detect regressions and new heavy queries.
Do	• Switch the slice to **SQL**. • Look at the top 5 by load. For each, note the SQL ID, what it does and which waits it is made of. • Compare with yesterday (DA-05). • Cross-check the statement in the database (code below).

Cross-check	`SELECT queryid, calls, round(mean_exec_time::numeric, 2) AS mean_ms,`
	`round(total_exec_time::numeric / 1000) AS total_s,`
	`shared_blks_read, left(query, 70) AS q`
	`FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;`

Healthy looks like	The same queries as usual with similar share of load and similar average latency.
If not	A new query in the top 5 or a known one with a much higher share: tune it (W-04 in the DBA guide, LV-01/LV-02 here). Note: some statistic column names differ by PostgreSQL version (`total_time` in version 12 and older).

DA-04	Find who is generating the load
Purpose	Identifies the application, user or host behind a spike, so you can talk to the right team.
Do	• Switch the slice to **Users**, then **Hosts**, then **Applications** (names depend on the engine and settings). • Match unusual entries to services, batch jobs, or a person running ad-hoc queries.

Live cross-check	`SELECT usename, application_name, client_addr, count(*) AS active_sessions`
	`FROM pg_stat_activity`
	`WHERE state = 'active' AND backend_type = 'client backend'`
	`GROUP BY 1, 2, 3 ORDER BY 4 DESC LIMIT 10;`

Healthy looks like	Load comes from the expected application accounts and hosts.
If not	An unknown host, an interactive user running heavy reports on production, or one application suddenly dominating: contact the owner and consider a separate replica or a role-level `statement_timeout` for ad-hoc users.

DA-05	Compare with a known-good period
Purpose	'Is this normal?' is only answerable against a baseline. Compare the same window today with yesterday or last week.
Do	• Use the time-comparison option in the dashboard, or open two windows with the same hours on different days. • Compare total load, wait mix and top SQL. • Remember to compare like with like (weekday versus weekday, same business event).
Healthy looks like	Same shape and mix. Differences are explained by known events.
If not	Unexplained differences: find what changed (deployment, data growth, parameter change, new customer) and use RL-01 for release-related changes.

DA-06	Record a baseline
Purpose	Short-term data ages out (7 days in Standard). A few numbers kept over time make capacity planning and incident analysis possible even without long retention.

Command	`# average and peak DB load for the last 24 hours (1-hour points)`
	`aws pi get-resource-metrics --service-type RDS --identifier <DbiResourceId> \`
	`  --start-time $(date -u -d '-24 hours' +%FT%TZ) --end-time $(date -u +%FT%TZ) \`
	`  --period-in-seconds 3600 \`
	`  --metric-queries '[{"Metric":"db.load.avg"}]' \`
	`  --query "MetricList[0].DataPoints[].[Timestamp,Value]" --output table`

Record	Peak and average load, peak load divided by vCPU count, top 3 waits, top 5 SQL IDs. Store in a shared sheet or a small table (see the size-history example in the DBA guide).
Watch out	The API identifier is the `DbiResourceId` (`db-...`), not the instance name.

Weekly, release-day and monthly

WE-01	WEEKLY: Top-SQL tuning list
Purpose	Turn what you see into a prioritised backlog instead of reacting only to incidents.
Do	• Over a 7-day window, slice by SQL and list the 10 statements with the most load. • For each: waits, calls, average latency, owner, suggested fix (index, rewrite, caching, schedule change). • Review the list with the application team, fix the top two, and re-check next week.
Plan check	`EXPLAIN (ANALYZE, BUFFERS) <statement>;   -- on a clone or off-peak` `CREATE INDEX CONCURRENTLY idx_<table>_<cols> ON <schema>.<table> (<cols>);`
Healthy looks like	Each week the top of the list gets shorter or cheaper, and the total load for the same traffic goes down.
Watch out	Execution plans are not captured automatically for RDS PostgreSQL. Use `EXPLAIN` and consider `auto_explain` to log slow plans.

WE-02	WEEKLY: Capacity ratio (peak load per vCPU)
Purpose	Decide scaling from evidence. The ratio of peak DB load to vCPU count is a simple capacity indicator.
Do	• Take the weekly peak and the weekly average of DB load. • Divide each by the vCPU count. • Track the trend in your baseline sheet.
Interpret	Peak ratio below about 0.7: comfortable. Between 0.7 and 1: watch closely. Regularly above 1 with CPU as the main wait: tune or scale up. Above 1 with I/O or lock waits: fix those waits first, since more CPU will not help. (These are rules of thumb; calibrate to your system.)
Watch out	Do not scale the instance to hide a single bad query. Check Top SQL first.

WE-03	WEEKLY: Lock review (Advanced mode)
Purpose	Find recurring blocking patterns before they become outages. Lock analysis captures lock snapshots every 15 seconds and shows a lock tree of blockers and waiters.
Do	• Open the Lock analysis tab for the past 7 days. • Look for repeated blockers: the same query, table or job. • Raise a ticket for each repeated pattern.
Healthy looks like	No high-locking banner, and no repeated multi-level blocking chains.
If not	A 'High locking detected' banner (shown after lock snapshots across 15 consecutive minutes) means act now: see LV-03.

RL-01	RELEASE DAY: Before and after comparison
Purpose	Releases are the most common cause of new performance problems. Compare load right before and after the change.
Do	• Before: note the load level, wait mix and top SQL for a representative hour. • Deploy. • After 15, 60 and 240 minutes: compare the same views. • Look for new SQL IDs, a changed wait mix, and any rise in `Lock:*` or `IO:*`.

Quick check	`-- sessions by wait event right now (works any time, no console needed)`
	`SELECT coalesce(wait_event_type || ':' || wait_event, 'CPU') AS wait, count(*) AS sessions`
	`FROM pg_stat_activity`
	`WHERE state = 'active' AND backend_type = 'client backend'`
	`GROUP BY 1 ORDER BY 2 DESC;`

Healthy looks like	Load and mix are unchanged, or changed in the intended way.
If not	Tell the release owner immediately with the evidence (time, SQL ID, wait). Roll back or hot-fix; see LV-08.

MO-01	MONTHLY: Right-sizing, retention and cost
Purpose	Keep monitoring cost in proportion to its value and keep instance sizes honest.
Do	• Review the capacity ratio trend (WE-02) per instance. • Decide which instances need Advanced mode and which can stay on Standard. • Check the monitoring line items in the bill (Database Insights, CloudWatch Logs for Enhanced Monitoring).
Healthy looks like	Production has Advanced mode and sensible retention; dev/test uses Standard; no unexplained cost growth.
Watch out	Changing from Advanced to Standard reduces available history to 7 days; export what you need first.

PART 4 | Real-time scenarios: what to click, run, and fix

Each scenario starts from what the chart shows. Follow the drill-down in the dashboard, confirm in the database with the SQL, understand the meaning, then fix. Stabilise first (stop the damage), then find the root cause. Tell stakeholders before terminating sessions.

LV-01 CPU is the dominant wait and load is above Max vCPU [HIGH IMPACT]

Chart shows	Load climbs above the Max vCPU line and the bulk of it is the `CPU` colour. Users report slowness.
Drill down	• Slice by **SQL**: which statements carry the most CPU load? • Slice by **Users/Hosts**: is it one application or a new caller? • Check whether it started at a known time (deployment, job).

Confirm (SQL)	`SELECT pid, usename, now() - query_start AS runtime, left(query, 80) AS q`
	`FROM pg_stat_activity`
	`WHERE state = 'active' AND wait_event IS NULL AND backend_type = 'client backend'`
	`ORDER BY query_start LIMIT 15;  -- active with no wait = on CPU`

Meaning	More sessions want CPU than there are vCPUs, so they queue. Usual causes: a bad plan or missing index, a new heavy query, or a burst of concurrent requests.
Fix	• Cancel the worst offender (`pg_cancel_backend(pid)`) if it is safe. • Roll back a release that introduced the query. • Add the missing index with `CREATE INDEX CONCURRENTLY`, or rewrite the query. • If the load is genuine, add a read replica for reads or scale the instance up.
Prevent	Alarm on DB load (SU-06), review top SQL weekly (WE-01), and load-test releases.

LV-02 IO:DataFileRead dominates

Chart shows	A large `IO:DataFileRead` band, often with rising read latency in CloudWatch.
Drill down	• Slice by **SQL** and pick the statements with the most IO waits. • Check if they read large tables without an index.

Confirm (SQL)	`-- statements reading the most blocks from outside shared buffers`
	`SELECT queryid, calls, shared_blks_read, shared_blks_hit, left(query, 60) AS q`
	`FROM pg_stat_statements ORDER BY shared_blks_read DESC LIMIT 10;`
	
	`-- tables read mostly by sequential scans`
	`SELECT relname, seq_scan, seq_tup_read, idx_scan`
	`FROM pg_stat_user_tables ORDER BY seq_tup_read DESC LIMIT 10;`

Meaning	Sessions wait for pages that are not in memory. Common causes: sequential scans on big tables, a working set larger than shared buffers and OS cache, or a cold cache after restart/failover/restore.
Fix	• Add or fix indexes for the top statements; avoid `SELECT *` on wide tables. • Update statistics with `ANALYZE` if the planner now chooses scans. • Allow time for cache warm-up after a restart. • If the working set is truly larger than memory, scale the instance memory or partition the data.
Prevent	Review sequential-scan statistics weekly (W-02 in the DBA guide) and keep `autovacuum`/analyze healthy.

LV-03 Lock waits: Lock:transactionid or Lock:relation [HIGH IMPACT]

Chart shows	A red `Lock:*` band that grows quickly while CPU stays moderate; many sessions pile up and the application times out.
Drill down	• Advanced mode: open **Lock analysis**. The lock tree shows the blocking session at the root, how many sessions it blocks, its last SQL and how long it has blocked. • Standard mode: use the SQL below.

Confirm (SQL)	`-- root blockers: block others but are not blocked themselves`
	`SELECT DISTINCT b AS root_blocker_pid`
	`FROM (SELECT unnest(pg_blocking_pids(pid)) AS b FROM pg_stat_activity) x`
	`WHERE NOT EXISTS (SELECT 1 FROM pg_stat_activity y`
	`WHERE y.pid = x.b AND cardinality(pg_blocking_pids(y.pid)) > 0);`
	
	`SELECT pid, usename, state, now() - xact_start AS xact_age, left(query, 80) AS q`
	`FROM pg_stat_activity WHERE pid = <root_blocker_pid>;`

Meaning	A transaction holds locks others need. The root is often a session that is `idle in transaction` (not counted in DB load because it is not active, so it can hide behind the waiters), a long batch job, or a DDL statement queued behind a long query with every later query queued behind it.
Fix	• Identify the root blocker and its owner. • Cancel it with `pg_cancel_backend(pid)`; terminate with `pg_terminate_backend(pid)` if needed. • If a queued DDL is the cause, cancel the DDL. • Tell the application team so the root cause is fixed.
Prevent	Use `lock_timeout` for DDL, keep transactions short, set `idle_in_transaction_session_timeout`, and run DDL off-peak.

LV-04 Client:ClientRead is large but the database is not busy

Chart shows	Load is moderate and mostly `Client:ClientRead`; the application is slow.
Drill down	• Slice by **Hosts/Users** to see which application is involved. • Check application logs and connection-pool metrics for the same period.

Confirm (SQL)	`SELECT state, count(*) FROM pg_stat_activity`
	`WHERE backend_type = 'client backend' GROUP BY state ORDER BY 2 DESC;`
	
	`SELECT pid, usename, now() - xact_start AS xact_age, left(query, 60) AS last_query`
	`FROM pg_stat_activity WHERE state = 'idle in transaction' ORDER BY xact_start LIMIT 10;`

Meaning	The database finished its part and is waiting for the client. The delay is usually in application code, a chatty network round-trip pattern, or transactions kept open while the application does other work.
Fix	• Do not tune the database for this: raise it with the application owner. • Reduce round trips (batch statements); keep transactions short. • Check network latency between application and database.
Prevent	Set `idle_in_transaction_session_timeout`, review ORM transaction handling.

LV-05 LWLock contention (lock_manager, buffer_content, others)

Chart shows	`LWLock:*` colours grow with concurrency; CPU is high but throughput stops improving.
Drill down	• Slice by **Waits** and identify the exact LWLock event. • Slice by **SQL** to see the statements involved.

Confirm (SQL)	`SELECT wait_event, count(*) FROM pg_stat_activity`
	`WHERE wait_event_type = 'LWLock' GROUP BY 1 ORDER BY 2 DESC;`
	
	`SELECT count(*) AS connections FROM pg_stat_activity WHERE backend_type = 'client backend';`

Meaning	Internal lightweight locks protect shared memory structures. High contention appears with very high concurrency, queries that touch many partitions or indexes (`lock_manager`), or hot pages (`buffer_content`).
Fix	• Reduce concurrency with a connection pooler (RDS Proxy or PgBouncer). • Reduce the number of partitions/indexes a query touches. • Spread hot writes across pages or tables. • If it persists, scale up and involve AWS Support with the Database Insights evidence.
Prevent	Connection budget per application and load tests at expected concurrency.

LV-06 Temp-file and WAL waits: IO:BufFileRead/Write and IO:WALWrite

Chart shows	`IO:BufFileRead/BufFileWrite` (temporary files) or `IO:WALWrite` bands, with write latency or temp-file growth in CloudWatch.
Drill down	• Slice by **SQL**: which statements spill or commit heavily?

Confirm (SQL)	`SELECT datname, temp_files, pg_size_pretty(temp_bytes) AS temp_bytes`
	`FROM pg_stat_database ORDER BY temp_bytes DESC;`
	
	`SELECT queryid, temp_blks_written, wal_bytes, left(query, 60) AS q  -- wal_bytes: PG 13+`
	`FROM pg_stat_statements ORDER BY temp_blks_written DESC LIMIT 10;`

Meaning	BufFile waits mean sorts or hashes exceed `work_mem` and spill to disk. WALWrite means very frequent commits or slow storage for the WAL.
Fix	• Temp files: add indexes that avoid the sort, or raise `work_mem` for the specific role or session (not globally). • WAL: batch many small commits into fewer transactions; check provisioned IOPS and write latency; ask whether `synchronous_commit` can be relaxed for non-critical writes.
Prevent	Alarm on write latency and review queries that write temp files (`log_temp_files`).

LV-07 Sudden connection storm

Chart shows	Load jumps within a minute; many sessions are active at once; connection count in CloudWatch spikes.
Drill down	• Slice by **Hosts** and **Users**: one source usually dominates.

Confirm (SQL)	`SELECT client_addr, usename, application_name, count(*) AS sessions`
	`FROM pg_stat_activity WHERE backend_type = 'client backend'`
	`GROUP BY 1, 2, 3 ORDER BY 4 DESC LIMIT 10;`
	
	`SHOW max_connections;`

Meaning	Often an application restart or retry loop that opens many new connections, or a traffic burst without pooling. Each connection uses memory and adds contention.
Fix	• Throttle or stop the offending client. • Terminate its idle sessions if the limit is near. • Introduce or resize connection pooling.
Prevent	RDS Proxy or PgBouncer, per-role `CONNECTION LIMIT`, and exponential backoff in application retries.

LV-08	Problem started right after a deployment
Chart shows	The load step-changes at the deployment time. A new SQL ID appears near the top or an existing one gets much heavier.
Drill down	• Use the time slider to select the 15 minutes after the change. • Slice by **SQL**, then compare with the 15 minutes before (DA-05).

Confirm (SQL)	`SELECT queryid, calls, round(mean_exec_time::numeric, 2) AS mean_ms,`
	`  rows / nullif(calls, 0) AS rows_per_call, left(query, 70) AS q`
	`FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;`

Meaning	A new or changed query (missing index, N+1 pattern, larger result set) or changed settings. The step-change timing is strong evidence.
Fix	• Share the SQL ID, text and chart with the release owner. • Roll back, feature-flag off, or hot-fix the query. • After the fix, confirm the load returns to the baseline.
Prevent	Pre-release load tests and a standard before/after check (RL-01).

LV-09	Intermittent slowness that other tools average away
Chart shows	Short spikes lasting seconds to a few minutes that 1-minute CloudWatch averages smooth out.
Drill down	• Zoom the time range to the incident window; the dashboard samples active sessions every second, so brief spikes remain visible. • Slice by **Waits** and **SQL** for just that window.

Catch it live next time	`-- run in psql during a slowdown; repeats every 5 seconds`
	`SELECT coalesce(wait_event_type || ':' || wait_event, 'CPU') AS wait, count(*) AS sessions`
	`FROM pg_stat_activity`
	`WHERE state = 'active' AND backend_type = 'client backend'`
	`GROUP BY 1 ORDER BY 2 DESC`
	`\watch 5`

Meaning	Typical causes: a scheduled job, a checkpoint/vacuum burst, or periodic lock contention. Correlate the timestamps with cron jobs and batch schedules.
Fix	• Match the spike time to the job or event that causes it, then move, throttle or tune it. • If periodic with a fixed interval, check scheduled jobs first.
Prevent	Keep job schedules documented and stagger heavy jobs.

LV-10	Load is low but users say it is slow
Chart shows	DB load is well below Max vCPU and the wait mix looks normal.
Drill down	• Confirm with application timing: where does the request time go? • Check `Client:ClientRead`, network latency, connection-pool wait, and downstream services.

Confirm (SQL)	`-- are there long-running queries at all?`
	`SELECT count(*) FILTER (WHERE now() - query_start > interval '5 seconds') AS over_5s,`
	`  count(*) AS active`
	`FROM pg_stat_activity WHERE state = 'active' AND backend_type = 'client backend';`

Meaning	If the database is not loaded and no query is slow, the delay is outside it: the application, the connection pool, DNS, the network, or a dependency.
Fix	• Hand over the evidence (flat DB load, normal waits, no slow queries) to the application and network teams. • Measure connection time from the application host.
Prevent	End-to-end tracing so the slow component is visible without guessing.

PART 5 | Problems with the tool, quick answers, cheat sheet

Troubleshooting the dashboard itself

Problem	What to check
The dashboard shows no data or a gap	Check the mode and that Performance Insights/Database Insights is enabled on the instance (SU-02). Gaps can follow a reboot or maintenance. Very small or unsupported instance classes may not support all features; check the AWS support list for your engine and class.
Top SQL shows statements but the text is cut off	The text length is limited by `track_activity_query_size` (static; reboot required). Increase it in the parameter group (SU-03) and reboot in a window.
Per-query statistics are empty	Confirm `pg_stat_statements` is in `shared_preload_libraries` and that `CREATE EXTENSION pg_stat_statements;` was run in the database.
I lost history beyond 7 days after the change	Instances that used paid Performance Insights retention but were moved to Database Insights Standard keep their configured retention, per AWS's notice; instances that fall back to the free level keep 7 days. If you need long history, use Advanced mode and export key data via the API.
I cannot find execution plans for my RDS PostgreSQL instance	Plan capture is offered for Aurora PostgreSQL, RDS for Oracle and RDS for SQL Server, not RDS for PostgreSQL. Use `EXPLAIN (ANALYZE, BUFFERS)` and `auto_explain`.
Lock analysis tab is missing or empty	It requires Advanced mode. If it is enabled and empty, there may simply have been no blocking in the period (snapshots are taken every 15 seconds).
Access denied for a user	Attach the managed policy `AmazonRDSPerformanceInsightsReadOnly` (or a custom policy that allows the needed `pi:` and `cloudwatch:` actions), and check any restrictions on SQL text.
A runbook says 'open Performance Insights'	The old console entry now redirects to CloudWatch Database Insights. Update the runbook text but keep the concepts: DB load, waits, SQL, users, hosts.
Costs went up	Advanced mode is billed per vCPU-hour, and Enhanced Monitoring and CloudWatch Logs add cost. Review per instance; use Standard for non-production.

Quick answers

Question	Answer
What is 'DB load' in simple words?	Average Active Sessions (AAS): on average, how many sessions are active (running or waiting) at the same moment. If it is 4, then on average 4 sessions are busy. It is sampled every second and split by what each session was doing.
Why compare it with the vCPU count?	Only as many sessions can run on CPU at the same time as there are vCPUs. If CPU-wait sessions exceed that number, they queue. Non-CPU waits (I/O, locks) do not use CPU, so a high load made of those points to a different bottleneck.
Does idle-in-transaction count in DB load?	Generally no: only active sessions are counted. That is why a blocker that is idle in transaction may not appear as load, while the sessions it blocks do. Use Lock analysis or `pg_stat_activity`.
What replaced Performance Insights?	CloudWatch Database Insights. The Performance Insights console experience ended on 31 July 2026 and now redirects there. The concepts (DB load, waits, top SQL) and the API remain.
What do I get free?	Standard mode, free for up to 7 days of detailed database and per-query metrics. Longer retention in Standard is paid, and Advanced mode is billed per vCPU-hour. Check the current pricing page for your Region.
Is there a performance cost to enabling it?	It is designed to be lightweight (it reads session state, no agent on your database). Enable it on production without fear, and measure on a clone if you have a very tight system.
Standard or Advanced for production?	Advanced if you want months of history, guided lock analysis, OS-process view and fleet monitoring. Standard if live troubleshooting with 7 days of history is enough.
Can I use it without the console?	Yes. The Performance Insights API (`aws pi ...`) still works for scripting, exports and custom dashboards, using the instance's `DbiResourceId`.
Pg_stat_statements or Database Insights: which one?	Both. Database Insights shows load over time sliced by waits, SQL, user and host; `pg_stat_statements` gives exact cumulative statistics per query. Use the dashboard to find when and what, and `pg_stat_statements` to confirm numbers.
Does it work on read replicas?	Enable it on each instance you want to observe, including replicas; each has its own load.

Cheat sheet

Console and API

Item	Use
`CloudWatch > Database Insights`	Dashboard: DB load, waits, SQL, users, hosts, (Advanced) locks, OS processes
`aws pi get-resource-metrics`	Time series (for example db.load.avg), optionally grouped
`aws pi describe-dimension-keys`	Top items by a dimension (SQL, waits, users, hosts)
`aws rds describe-db-instances`	Mode, retention, DbiResourceId
`aws rds modify-db-instance --database-insights-mode advanced|standard`	Switch mode

Dimension groups used by the API

Item	Use
`db.wait_event`	Slice load by wait event
`db.sql_tokenized`	Slice load by normalised SQL
`db.user`	Slice load by database user
`db.host`	Slice load by client host

Live SQL that mirrors the dashboard

Item	Use
`SELECT coalesce(wait_event_type||':'||wait_event,'CPU'), count(*) FROM pg_stat_activity WHERE state='active' AND backend_type='client backend' GROUP BY 1 ORDER BY 2 DESC;`	Current wait mix (your own mini load chart)
`SELECT queryid, calls, mean_exec_time, total_exec_time FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;`	Heaviest statements
`SELECT pg_blocking_pids(pid) FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid))>0;`	Who is blocked and by whom

A 5-minute triage you can run from memory

1. Open Database Insights for the instance and set the window to the incident time.
2. Is load above Max vCPU? Which colour is biggest?
3. Slice by SQL: write down the top statement and its SQL ID.
4. Slice by Users and Hosts: who sent it?
5. Compare with the same time yesterday: is this new?
6. Confirm with `pg_stat_activity` and `pg_stat_statements`, then act using the matching LV scenario.